الأكاديمية السعودية الرقمية
SAUDI DIGITAL ACADEMY

WeCloudData

PROJECT REPORT



JobDataPipeline

SUBMITTED BY : TEAM C

MEMBERS :

GHALA ALSALEM
MURADI ALMUTAIRI
MOHAMMED ALHARBI
LAMIA ALSUHIBANI

CONTENTS

EXECUTIVE SUMMARY..... 3
PROBLEM STATEMENT..... 3
PROPOSED SOLUTION..... 4
TARGET STAKEHOLDERS..... 5
DATA SOURCES..... 6
DATA COLLECTION..... 6
CHALLENGES ENCOUNTERED..... 8
PIPELINE ARCHITECTURE..... 9
ORCHESTRATION 11
DATA TRANSFORMATION..... 12
DATA MODEL..... 15
DATA QUALITY APPROACH..... 20
RESULTING DATASET..... 22
DOWNSTREAM APPLICATIONS..... 25
KNOWN LIMITATIONS..... 28
SUMMARY OF THE RESULT..... 28

1. EXECUTIVE SUMMARY

The Job Data Pipeline project is designed to automate the collection and processing of technology job market data in Saudi Arabia from multiple sources. The pipeline transforms raw technology job postings into a structured and analysis-ready dataset by applying data cleaning, standardization, translation, and integration processes

The resulting dataset supports the analysis of in-demand tech skills, technology job ,demand, job locations, companies, job roles, and market trends across Saudi Arabia providing reliable data for generating meaningful insights and supporting data-driven decisions about the Saudi technology job market

2. PROBLEM STATEMENT

2.1. Current Situation

The technology job market generates a large volume of data across multiple sources. However, this information is often fragmented, inconsistent, and available in different formats and languages, making it difficult to obtain a clear and reliable view of current market demand.

2.2.Key Challenges

- Job data is distributed across multiple sources with varying structures and formats.
- Inconsistent job titles, skills, locations, and company information reduce data comparability.
- Missing, duplicate, and outdated records affect data quality.
- Multilingual job postings create additional challenges in data standardization.
- The lack of centralized, structured data makes it difficult to efficiently identify market trends and in-demand skills.

2.3. Impact

These challenges increase the effort required to collect and analyze job market data and can lead to incomplete or unreliable insights. Without a consistent and up-to-date data foundation, organizations and researchers may face difficulties in accurately understanding technology employment trends and emerging skill demands.

3. PROPOSED SOLUTION

3.1 SOLUTION OVERVIEW

The proposed solution is a single, automated data pipeline that solves the fragmentation problem described in Section 2. It runs on a schedule, applies the same rules every time, and only lets validated records into the final dataset — making it automated, repeatable, and reliable.

3.2 WHAT WE ARE BUILDING

The core of the solution is one pipeline that automatically connects to five selected sources — Jooble, JSearch, FreeHire, Tapneo, and Tanqeeb — collecting new postings on an ongoing basis rather than as a one-time effort. From the start, it is built to handle the real-world issues already observed in the raw data: sources structure their postings differently, some records are missing key fields, the same job often appears on more than one platform, and content can be in Arabic or English. Handling these issues is part of the pipeline's core design, not an afterthought.

3.3 WHAT IT DELIVERS

As data moves through the pipeline, it passes through three layers: Bronze for raw data, Silver (staging) for cleaned and standardized records, and Gold (mart) for the final, merged dataset.

The extraction and initial cleaning process runs automatically on a scheduled basis with no manual intervention, handling raw job postings, mapping them into a common structure, and removing duplicate listings. Following this, data transformation and modeling are applied using dbt, ensuring every record is validated and transformed into a reliable, structured, and analysis-ready dataset — serving as a solid foundation for dashboards and market analysis.

4. TARGET STAKEHOLDERS

4.1 PRIMARY USERS: JOB SEEKERS & TECH TALENT

Fresh graduates and professionals seeking opportunities in the Saudi technology sector. The platform provides them with actionable insights into top demanded tech skills, trending job categories, and major hiring companies to guide their career development.

4.2 SECONDARY USERS: TECH RECRUITERS & HR PROFESSIONALS

Hiring teams and recruiters looking to optimize their talent acquisition strategies within the local market. The platform enables them to benchmark role requirements, track market demand, and identify skill availability and top hiring companies across the Saudi tech ecosystem.

4.3 REGIONAL & LABOR MARKET ANALYSTS

Data analysts, researchers, and stakeholders who evaluate regional workforce distribution. The platform enables them to analyze the geographic distribution of tech jobs in Saudi Arabia, identify localized hiring concentrations across regions, and evaluate technology market trends and employment patterns.

5. DATA SOURCES

The pipeline draws on five distinct sources to ensure broad and reliable coverage of the Saudi Arabian tech job market, combining official APIs, web scraping, and a static historical dataset.

Jooble is accessed through its official API, using an authenticated POST request with a custom API key. Data is returned in JSON format, providing structured, reliable job listings.

JSearch is accessed via RapidAPI using a standard GET request. It returns data in both CSV and JSON formats and offers broad international job coverage, including postings relevant to the Saudi market.

FreeHire is accessed through a direct API request made via the browser, returning JSON-formatted job data from its endpoint.

Tapneo differs from the other sources in that it is not queried live. Instead, a static dataset is downloaded once and read directly using Pandas. This source serves as a historical baseline rather than an ongoing feed, since no live extraction script exists for it.

Tanqeeb, specifically the Saudi Arabia section of the site, does not provide a public API. Data from this source is therefore collected through web scraping, using BeautifulSoup in combination with a ScrapeOps proxy to handle JavaScript rendering and avoid blocking. Data returned is semi-structured JSON.

6. DATA COLLECTION & INGESTION APPROACH

6.1. Overview

Data is collected by dedicated Python scripts for the four live sources: REST API calls for Jooble, JSearch, and FreeHire, and a scraper for Tanqeeb. Tapneo is imported manually as a one-time load. The Jooble and FreeHire scripts retry with exponential backoff on transient server errors (429, 500, 502, 503, 504), while JSearch and Tanqeeb log failures and skip to the next request.

Raw data is first written to Azure Data Lake Storage Gen2, organized by source and ingestion date (e.g., raw/<source>/ingest_date=<date>/data.json). This preserves the original data for auditability and reprocessing.

From there, data is loaded into Snowflake with COPY INTO, populating a raw (Bronze) layer with one table per source (e.g., RAW_JOOBLE_JOBS). Snowflake tracks which files were already loaded, so re-runs do not create duplicates.

6. DATA COLLECTION & INGESTION APPROACH

6.2. Ingestion Pattern

The pipeline follows a pull-based batch model: each source is queried on demand or on a schedule, rather than receiving continuous pushes. This was chosen because:

- All sources expose request/response APIs or scrapeable pages, not event streams or webhooks.
- Job postings are low-frequency data, so batch pulls capture new records without unnecessary API usage.
- Batch ingestion fits the Bronze-Silver-Gold model, where each run produces a traceable, timestamped raw snapshot.

6.3. Pagination Handling

Each source is queried with a strategy suited to its API and rate limits:

Source	Pagination Type	Mechanism
Jooble	Query fan-out (no page increment)	Always requests `page` 1, iterating through combinations of 16 Saudi cities × 18 tech keywords until 50 valid jobs are collected
JSearch	Multi-page per request	`num_pages` is set to 4, so each call returns several pages; loops through 30 keywords until 100 new jobs are collected
FreeHire	Offset-based	`limit` (20) and `offset` parameters control the result window, with the offset incremented per iteration until 150 jobs are collected or results run out
Tanqeeb	Page-based (scraped)	Sequential page numbers embedded in the scraped URL, up to 100 pages, until 20 new listings are collected
TapNeo	Not applicable	Delivered as a single bulk file, retrieved in one download

6.4. Error Handling and Rate Limiting

- Each source stops once a fixed target job count is reached, which caps requests per run, and a short delay (0.5–2 seconds) is applied between consecutive requests to avoid triggering rate limits.
- The Jooble script retries with exponential backoff on 429 and 5xx errors, and FreeHire on 5xx errors. JSearch logs a warning when the RapidAPI limit is reached and skips that keyword, and Tanqeeb logs failures and moves on.
- HTTP status codes are checked explicitly, and failed requests are skipped instead of stopping the run.

7. CHALLENGES ENCOUNTERED

7.1 DATA EXTRACTION CHALLENGES

During the data ingestion phase, the pipeline encountered rate limit constraints and quota restrictions imposed by external APIs (such as JSearch and Jooble). To mitigate potential connection interruptions and prevent exceeding the maximum allowed requests per second or day, intelligent delays and throttling mechanisms were implemented between requests, ensuring a smooth and uninterrupted data collection process.

7.2 DATA CLEANING & MODELING

- **TapNeo's** historical file did not include a job title field, so titles were reconstructed from the job description text.
- **Tanqeeb**, a localized Saudi platform, returned some listings in Arabic, which required a language check before enrichment.
- **Jooble** returns a truncated text snippet that varies slightly on every crawl, causing the same job to appear as duplicates when comparing descriptions. This was resolved by deduplicating on job title instead — a partial fix.
- Merging job postings and skills into **one fact table** would have caused a fan-out, inflating posting counts by the average number of skills per job. This was resolved with a Fact Constellation: two fact tables (fct_job_postings, fct_job_skills) sharing conformed dimensions.
- Skill name variants were also unified through a skill_synonyms seed, and missing values in key fields were preserved using an Unknown Member pattern instead of being dropped.

7.3 PIPELINE ORCHESTRATION CHALLENGES

Two operational issues were encountered when automating the pipeline with Prefect. First, Prefect could not access our GitHub repository by default, as the pipeline code is hosted in a private repository; this was resolved by generating a GitHub personal access token and configuring Prefect with the required permissions. Second, the pipeline code was originally structured to run from a local file path on a single machine, which is not compatible with automated, repeatable execution. This was resolved by configuring the Prefect flow to pull the pipeline code directly from the GitHub repository at runtime, decoupling execution from any single local environment.

8.PIPELINE ARCHITECTURE

8.1 DESIGN PRINCIPLE

The pipeline follows a Medallion Architecture, moving data through three progressively refined layers — Bronze (raw), Silver (staging), and Gold (marts) — with each layer serving a single, well-defined purpose and no layer being modified in place by a later stage.

8.2 EXTRACTION & LANDING ZONE

Each of the four live sources (Jooble, JSearch, FreeHire, Tanqeeb) has a dedicated Python extractor that retries automatically on transient failures. Extracted files are uploaded as JSON to Azure Data Lake Storage Gen2, partitioned by source and ingestion date. TapNeo is the exception to this path: as a one-time historical file rather than a live source, it was loaded directly into the raw layer without passing through this extraction

8.3 LOADING INTO SNOWFLAKE

New files are loaded from the data lake into source-specific RAW tables in Snowflake using a COPY INTO operation. Snowflake tracks previously loaded files at the storage-integration level, so re-running this load step does not reprocess or duplicate files already ingested.

8.4 INITIAL CLEANING

The initial data preprocessing pipeline performs several core operations. It begins with advanced filtering and non-tech removal by inspecting job titles and descriptions to ensure they are legitimate tech roles, filtering non-tech engineering fields (such as civil or architectural engineering that may occasionally appear by mistake). Following this, it handles extraction and parsing by automatically extracting and mapping technical skills (such as Python, SQL, React, AWS, Docker, etc.) from unstructured descriptions using extensive keyword dictionaries and Regular Expressions, alongside extracting educational requirements where available.

8.PIPELINE ARCHITECTURE

We chose Python mainly because our preprocessing involved heavy text processing. For example, we needed to extract technical skills from the job description, identify whether a job was actually technical, and handle different ways of writing the same skill or technology. These tasks were more manageable and maintainable in Python.

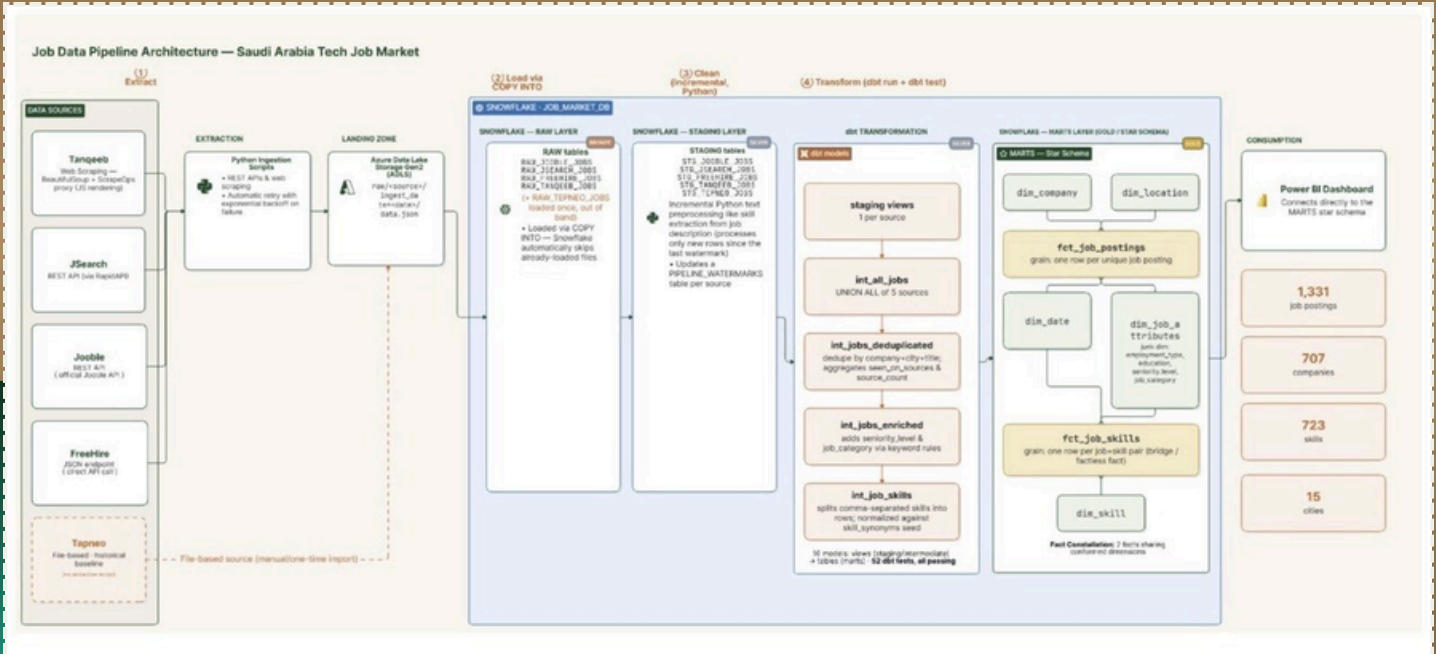
Python step reads only new rows since each source's last watermark, maps all sources into a unified schema, and updates the watermark—making this stage fully incremental.

8.5 TRANSFORMATION WITH DBT

dbt unions all sources, removes cross-source duplicates, enriches records with seniority and job category, and splits skills into normalized rows. The final marts layer is a Star Schema, validated by an automated suite of dbt tests.

8.6 CONSUMPTION

Power BI connects directly to the marts layer to power the analytics dashboard, providing live, always-current counts of job postings, companies, skills, and cities as the underlying data changes.



9.ORCHESTRATION

9.1 TOOL SELECTION

Prefect was selected to automate the pipeline's extraction, loading, and cleaning stages. As a Python-native orchestration framework, a Prefect task is a plain decorated function with no separate DAG file, scheduler service, or metadata database to configure — an appropriate trade-off for a pipeline of this scope compared to heavier alternatives such as Airflow.

9.2 FLOW DESIGN

The orchestrator is implemented as a single Prefect flow composed of tasks across three stages. In Stage 1 (Extract), one task runs per live source (Jooble, JSearch, FreeHire, Tanqeeb). In Stage 2 (Load), a single task loads all RAW tables via COPY INTO. In Stage 3 (Clean), one task per source standardizes it into its STAGING table. Because tasks are invoked directly within the flow rather than submitted asynchronously, execution within each stage is sequential rather than concurrent, following a fixed source order. The wait_for parameter is used between stages to make the dependency explicit — Load only begins once all extraction tasks have completed, and each cleaning task only begins once the load has completed.

9.3 RELIABILITY

Every task in the flow is configured with a single retry and a fixed delay before retrying, allowing transient failures — such as a slow API response or a temporary rate limit — to self-recover without failing the entire run.

9.4 EXECUTION ENVIRONMENT

The flow is configured to pull the pipeline source code directly from the project's GitHub repository at runtime using an authenticated access token, rather than executing from a fixed local path, allowing the flow to run consistently regardless of the machine that triggers it.

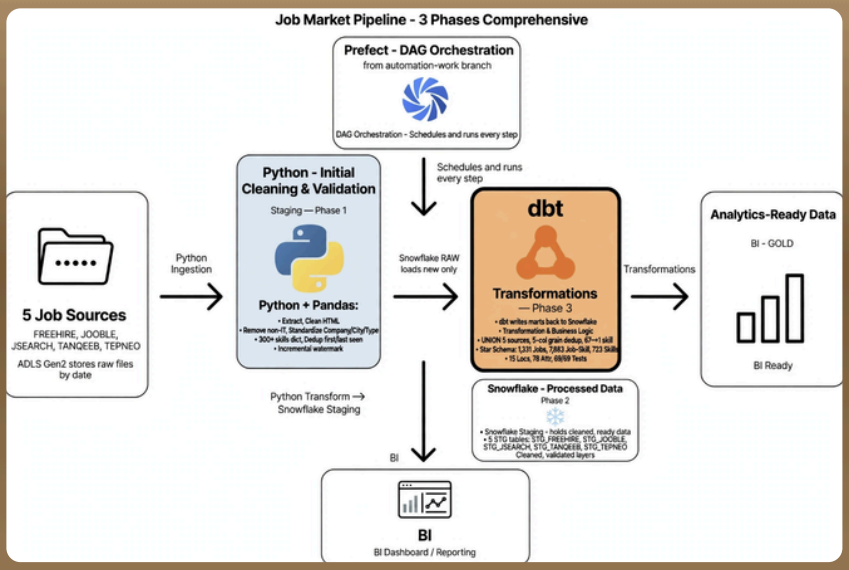
9.5 RELATIONSHIP TO THE TRANSFORMATION LAYER

dbt execution (dbt run and dbt test) is intentionally kept outside the Prefect flow and is run as a separate step once the flow completes. This separates data movement (Prefect's responsibility) from data modeling (dbt's responsibility), so a change to a dbt model can be re-run independently without re-triggering extraction or loading.

10. DATA CLEANING & TRANSFORMATION

The data cleaning and transformation phase is a fundamental component of the job pipeline project, aiming to convert job data—sourced from various origins and characterized by differing formats and quality levels—into standardized, organized, and analysis-ready data.

This phase was implemented across several sequential stages rather than performing all cleaning operations in a single step; the data is processed incrementally, moving from the extraction stage through to the final model in Snowflake.



The Goal of the Data Cleaning & Transformation Process

The data used in the project is collected from five different job listing sources:

- **Jooble**
- **JSearch**
- **Freehire**
- **Tanqeeb**
- **Tapneo**

These sources differ in how they represent data; variations exist in field names, date formats, the spelling of company and city names, and the presentation of skills and job descriptions.

10. DATA CLEANING & TRANSFORMATION

Therefore, it was necessary to perform cleaning and transformation operations to:

1. Standardize the format of data coming from different sources.
2. Remove or handle unsuitable values.
3. Standardize city and company names.
4. Clean text and job descriptions.
5. Extract technical skills from the data.
6. Standardize various skill formats.
7. Standardize job types and educational qualifications.
8. Identify non-technical jobs and exclude them from the project's data scope.
9. Handle duplicates across different sources.
10. Add derived information, such as experience level and job classification.
11. Prepare the data for the data modeling process within dbt

10.1 PHASE 1: INITIAL CLEANING USING PYTHON

The initial cleaning phase was carried out using Python and Pandas .
This phase focused on preliminary cleaning and data organization, while transformations related to final data logic were delegated to Snowflake and dbt.

1. Technical Scope Verification:

Since the project focuses on technical roles, a verification mechanism was established to determine whether a given role is technical in nature.

2. Cleaning and Processing Key Fields:

Cities (Location): Standardizing name variations across different sources.
Employment Type and Qualifications: Parsing descriptions to extract employment type (e.g., full-time) and educational qualifications, then standardizing their formats.

10. DATA CLEANING & TRANSFORMATION

3. Cleaning Job Titles:

Job titles vary across sources and may contain extraneous information. Rules were applied to extract the appropriate title and handle non-technical cases; a default value is used if no suitable title is found. A list of common technical roles was defined, including Data Engineer, Data Analyst, Data Scientist, Software Engineer, Cloud Engineer, DevOps Engineer, etc.

4. Cleaning Job Descriptions:

HTML tags, HTML entities, repetitive phrases, symbols, and irrelevant information were removed. Whitespace and line breaks were standardized, and repetitive headers—such as "Job Description," "Job Summary," "About the Role," "Job Purpose," and "Required Qualifications"—were eliminated.

5. Skill Extraction :

Skills: Extracting over 300 technical skills from the text using a dedicated dictionary.

10.2 PHASE 2: TRANSFORMATION IN SNOWFLAKE

Following initial cleaning in Python, data is transferred to Snowflake into Staging layers.

11. DATA MODEL & DESIGN DECISIONS

11.1 TRANSFORMATION & CLEANING USING DBT

The **dbt** phase begins to execute transformations based on data logic and build the final analytical model.

dbt is structured into three main layers:

Staging
↓
Intermediate
↓
Marts

Staging Layer :

In the Staging layer, a separate model was created for each source:

- **`stg_jooble_jobs`**
- **`stg_jsearch_jobs`**
- **`stg_freehire_jobs`**
- **`stg_tanqeeb_jobs`**
- **`stg_tapneo_jobs`**

The function of this layer is to standardize the format of data coming from the various sources.

Intermediate Layer

The intermediate layer contains the core transformation logic.

1. Merging sources: **`int_all_jobs`** uses **`UNION ALL`** to combine the five sources while retaining all records; deduplication occurs later in a separate model.

11. DATA MODEL & DESIGN DECISIONS

2. **Cross-Source Deduplication:** **`int_jobs_deduplicated`** identifies the same job posting across multiple sources. The grain is defined by five fields—**`company`**, **`city`**, **`country`**, **`location`**, and **`title`**—grouped together while retaining attributes like **`seen_on_sources`**, **`source_count`**, **`description`**, and **`skills`**. We use **`LISTAGG(DISTINCT source_name)`**,(e.g., Jooble, JSearch, Tanqeeb) and `source_count=3` to preserve source information without loss.

3. Job Enrichment: **`int_jobs_enriched`** adds derived information:

- **Seniority Level:** Derived from the title → Senior / Junior / Management / Mid
- **Job Category:** Derived from the title → Data, Security, DevOps/Infrastructure, Software Engineering, Design, Other

4. `int_job_skills.sql` — Skill parsing and synonym standardization

This file transforms the **`skills`** column (a single text string containing multiple comma-separated skills) into a separate row for each skill. Here is a real-world example of the issue we resolved: without standardization, "Google Cloud" appeared in the data in three distinct forms (`gcp`: 43 times; `google cloud`: 17 times; `google cloud platform`: 7 times). Consequently, any analysis of the "most in-demand skill" would have obscured its true prevalence (67 total occurrences). We solved this using a seed file (`skill_synonyms.csv`) that maps each variation to a standardized name.

11. DATA MODEL & DESIGN DECISIONS

Marts Layer

This is the final, analytics-ready layer, utilizing a Star Schema built according to the Kimball methodology.

First: The Five Dimensions:

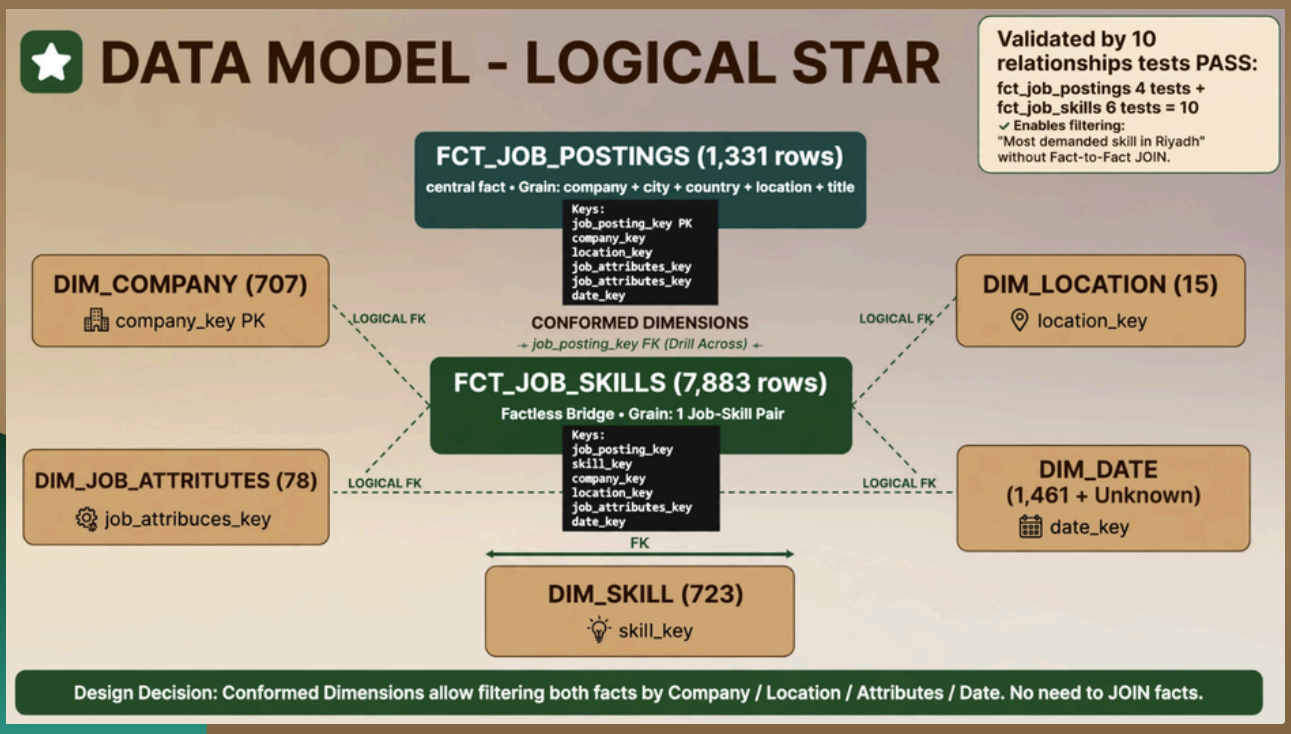
1. dim_company: One row per unique company .

2. dim_location: One row per city-country combination.

3. dim_date: The only dimension mathematically generated (without source data) using a **`date_spine`** ranging from 2024-01-01 to 2027-12-31 (1,461 days), including derived columns: year, month, and is_weekend.

4. dim_job_attributes – Junk Dimension (Design Decision): Why combine them? Because there were four attributes, each consisting of only a single column—a practice discouraged in Kimball modeling. We therefore consolidated **`employment_type`**, **`education`**, **`seniority_level`**, and **`job_category`** into a single dimension with a composite key to avoid having four separate single-column tables.

5. dim_skill: One row per skill following standardization (723 skills resulting from the consolidation of 67 synonyms, such as "GCP").



11. DATA MODEL & DESIGN DECISIONS

Second: The Two Fact Tables – Why Two Instead of One?

The pipeline uses two fact tables, each with a clearly defined and consistent grain:

- **fct_job_postings:**
  Grain = one row per unique job posting.
  It contains measures such as job_posting_count, source_count, and seen_on_sources.

- **fct_job_skills:**
  Grain = one row per job posting–skill combination.
  It follows the Factless Fact Table pattern, where each row represents a job posting associated with a specific skill.

Keeping these processes separate prevents skill-level records from multiplying job-level records and avoids inflated aggregations when counting job postings. The two fact tables share common dimensions, forming a Fact Constellation pattern.

11.2 Design Decisions

Key Design Decisions:

1. **Consistent job_posting_key :** The job_posting_key is generated using the same five attributes that define the job grain: company, city, country, location, and title. This ensures consistent identification and prevents potential key collisions between fct_job_postings and fct_job_skills.
2. **Conformed Dimensions :** Shared dimension keys (company_key, location_key, job_attributes_key, and date_key) are included in both fact tables. This allows skills to be analyzed directly by company, location, job attributes, and date without requiring direct fact-to-fact joins.
3. **Unknown Member Pattern for Missing Values :** Missing attributes are retained rather than removing incomplete records. Standardized values such as Unknown Company and Not Specified, along with an appropriate unknown date member, are used to preserve the complete job record while making missing information explicit and analyzable.

11. DATA MODEL & DESIGN DECISIONS

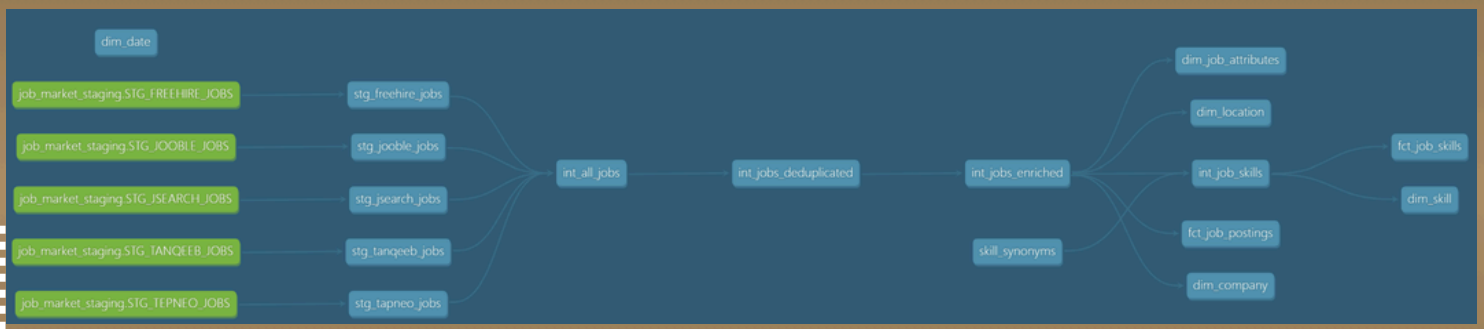
11.3 DATA LINEAGE – END-TO-END DATA FLOW

The pipeline follows a clear end-to-end lineage from the extracted source data to the final analytical fact and dimension tables.

The five tables loaded into Snowflake from Python flow into five **stg_*.sql** staging models, where columns are standardized without applying major business transformations. The data is then combined through **int_all_jobs** using UNION ALL, followed by **int_jobs_deduplicated**, which establishes the job grain using five attributes: company, city, country, location, and title.

The deduplicated data then flows into i**nt_jobs_enriched,** where additional business attributes such as seniority_level and job_category are derived. From this point, the data branches into the dimension models and the skill-processing path. The dimension models support **fct_job_postings**, while **int_job_skills** processes individual skills and their normalized synonyms before feeding **dim_skill** and **fct_job_skills**.

dim_date appears separately in the dbt lineage graph because it is generated using a date_spine rather than being derived through a ref() relationship from the source data. Its functional relationship with the fact tables is nevertheless established through the corresponding date keys and validated using successful relationships tests.



12. DATA QUALITY APPROACH

Data quality in the dbt layer is enforced through automated, declarative tests that run as part of every build, so a model is only considered complete once its data has been validated.

The tests are defined alongside the documentation in each layer's `schema.yml` and cover four areas.

Completeness: is checked with **`not_null`** tests on job titles, company names, and every surrogate and foreign key.

Uniqueness: is checked with **`unique`** tests on all dimension keys and on `job_posting_key`, plus a composite uniqueness test on the deduplicated model.

Referential integrity: is checked with ten **`relationships`** tests confirming that every foreign key in **`fct_job_postings`** and **`fct_job_skills`** resolves to a row in its dimension or parent fact.

Domain validity: is checked with an **`accepted_values`** test on employment type, set to warn severity so unexpected categories are surfaced without blocking the pipeline.

Instead of discarding incomplete records, missing company names, cities, employment types, education levels, and posting dates are mapped to explicit **"Unknown"** or **"Not Specified"** members. This preserves the record, keeps foreign keys valid, and makes the gaps measurable.

During development these tests exposed real defects, including orphaned company and date keys and a mismatch between the posting key and the deduplication grain, which were corrected at the source and re-verified.

12. DATA QUALITY APPROACH

After the latest refresh, which passed newly ingested postings through the full pipeline, a complete dbt build ran **1 seed, 9 views, and 7 tables** together with **52 data tests**. It finished in just **34.69s** with **69 of 69 checks passing**, with **0 warnings and 0 errors**.

The refreshed marts contain **1,331** job postings **(up from 1,256)**, **7,883** job–skill rows (up from 7,160), **707** companies, **723** skills, **78** job-attribute profiles, **15** locations, and a **1,461-**row date dimension. This shows that key uniqueness and referential integrity hold as data volume grows.

The **52 tests** break down as **33 not_null, 7 unique, 10 relationships, 1 accepted_values**, and **1 composite uniqueness test,** so you can cross-check them against the log.

```
09:00:03  69 of 69 START test relationships_fct_job_skills_skill_key__skill_key__ref_dim_skill_  [RUN]
09:00:04  69 of 69 PASS relationships_fct_job_skills_skill_key__skill_key__ref_dim_skill_  [PASS in 0.23s]
09:00:04
09:00:04  Finished running 1 seed, 7 table models, 52 data tests, 9 view models in 0 hours 0 minutes and 34
.69 seconds (34.69s).
09:00:04
09:00:04  Completed successfully
09:00:04
09:00:04  Done. PASS=69 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=69
```

13. RESULTING DATASET

13.1 FINAL CURATED DATASET

Following the ingestion, cleaning, standardization, deduplication, and validation stages, the pipeline produced a final curated dataset for the Saudi technical job market. The resulting data represents a structured and consistent version of job postings collected from multiple sources and is prepared for downstream analytics and applications.

The final dataset was generated on **September 27, 2026** and consists of seven curated mart tables. These tables provide job-level information together with supporting information about technical skills, companies, locations, job attributes, and dates.

The final curated dataset contains **1,331 unique job postings**, **707 companies, 15 locations**, and **723 distinct technical skills**. In addition, the **FCT_JOB_SKILLS table** contains **7,883 job-skill relationships**, representing the association between job postings and the technical skills extracted from their descriptions.

The resulting dataset therefore provides a reusable analytical layer rather than requiring downstream users to work directly with the different structures and formats of the original source data.

13.2 DATASET SUMMARY

Table	Description	Row Count
FCT_JOB_POSTINGS	Unique technical job postings	**1,331**
FCT_JOB_SKILLS	Job-to-skill relationships	**7,883**
DIM_SKILL	Distinct technical skills	**723**
DIM_COMPANY	Hiring companies	**707**
DIM_LOCATION	Job locations	**15**
DIM_JOB_ATTRIBUTES	Employment, education, seniority, and job category attributes	**78**
DIM_DATE	Calendar and date attributes	**1,461**

TABLE 11.1. SUMMARY OF THE FINAL CURATED DATASET

13. RESULTING DATASET

13.3 DATASET CONTENTS

- **Job Information**

  The curated dataset contains standardized job-level information, including job titles, companies, locations, employment types, education requirements, seniority levels, job categories, posting dates, and source information.

- **Technical Skills**

  Technical skills extracted from job descriptions are represented separately to support analysis of skill demand. The final dataset contains **723** distinct technical skills and **7,883** job-skill relationships. This distinction is important because a single job posting can require multiple technical skills.

- **Company Information**

  Company information is standardized into a dedicated company dataset containing **707** unique companies. This enables analysis of hiring activity across organizations.

- **Geographic Information**

  Location information is standardized into **15** locations, allowing the dataset to support geographic analysis of technical job opportunities within Saudi Arabia.

- **Job Attributes**

  Job attributes include employment type, education requirements, seniority level, and job category. These attributes allow job postings to be analyzed according to their characteristics rather than only by title or company.

- **Date Information**

  A dedicated date dataset is included to support time-based analysis using calendar attributes such as day, month, quarter, and year.

13. RESULTING DATASET

13.4 UNDERSTANDING THE JOB-SKILL RELATIONSHIPS

The relationship between job postings and technical skills is represented by **7,883** records in **FCT_JOB_SKILLS**. These records should not be interpreted as **7,883** unique skills. Instead, they represent **individual relationships between a job posting and a skill.** Since one job can require multiple skills, the number of job-skill relationships is expected to be greater than the number of unique job postings or unique skills.

For example, a single job posting may be associated with Python, SQL, AWS, and Docker. Each association is represented as a separate job-skill relationship, while each technology itself appears only once in the DIM_SKILL dimension.

13.5 FINAL DATASET READINESS

The final curated dataset is designed to be consumed by **downstream analytical applications** without requiring the application layer to repeat the source-specific cleaning and standardization processes. The data has already been structured into consistent entities and relationships, allowing downstream tools to focus on analysis and visualization.

This separation also makes the resulting dataset reusable. **For example,** the same curated data can be consumed by a business intelligence dashboard, a search application, an analytical model, or future AI-powered applications.

14. DOWNSTREAM APPLICATIONS

14.1 IMPLEMENTED DOWNSTREAM APPLICATION: INTERACTIVE DASHBOARD

As a downstream application of the curated dataset, an interactive **Power BI dashboard** was developed to provide an analytical view of the Saudi technical job market.

The dashboard consumes the curated **mart data** rather than the raw source tables. This allows the visualization layer to work with standardized and validated data produced by the data engineering pipeline.

The dashboard provides an overview of the job market through key indicators and visualizations covering **job postings, companies, technical skills, locations, job categories, and hiring company.**

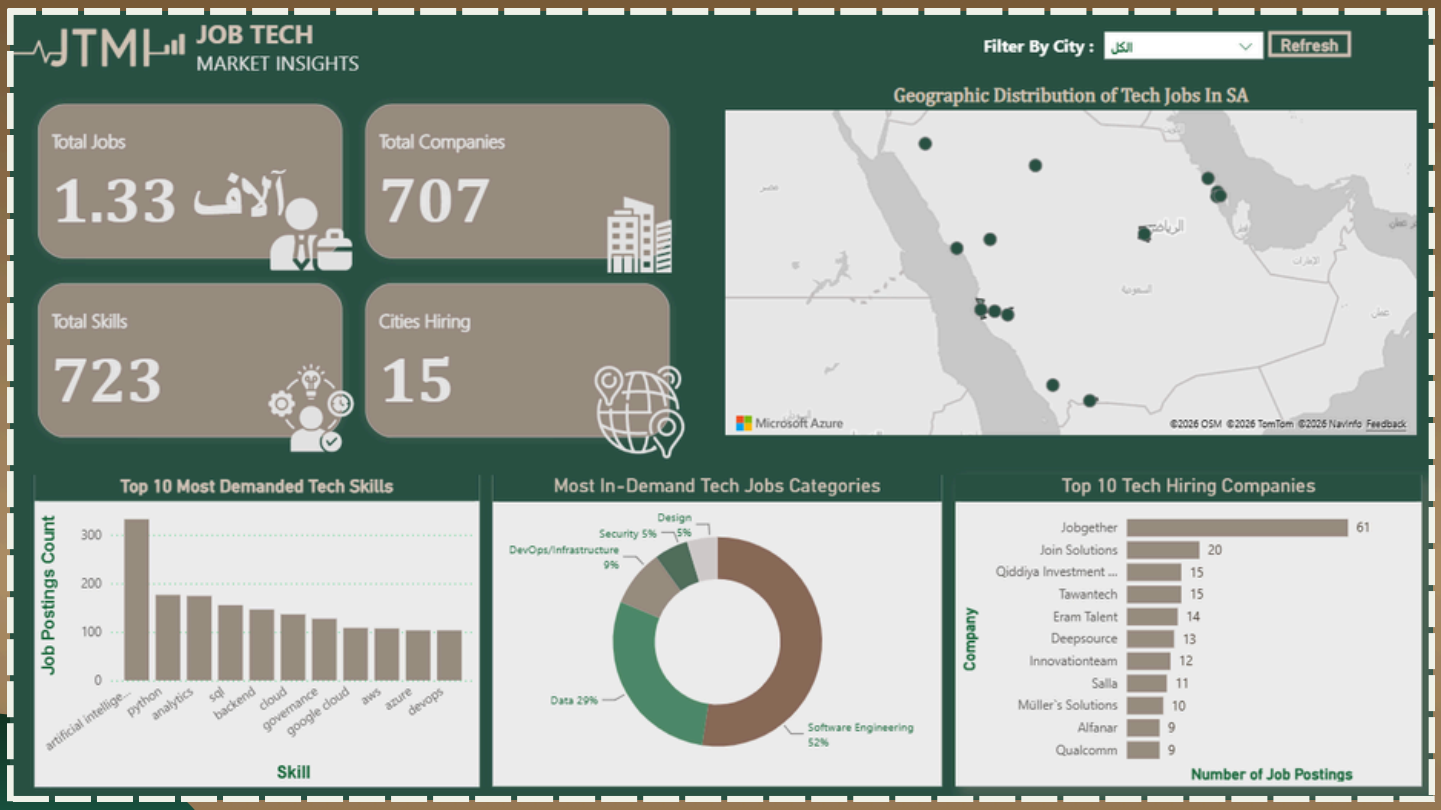


FIGURE 12.1 INTERACTIVE POWER BI DASHBOARD BUILT USING THE CURATED JOB-MARKET DATASET

14. DOWNSTREAM APPLICATIONS

14.2 DASHBOARD CAPABILITIES

- **Overall Job Market Overview**

  The dashboard provides high-level indicators for the number of job postings, companies, technical skills, and cities represented in the dataset. These indicators provide a quick overview of the size and coverage of the curated dataset.

- **Geographic Distribution**

  A geographic visualization shows the distribution of technical job postings across the identified Saudi locations. This allows users to explore where technical job opportunities are concentrated within the collected data.

- **Technical Skill Demand**

  The dashboard visualizes the most frequently occurring technical skills across job postings. This enables users to identify which technologies and technical competencies appear most frequently in the collected job postings.

- **Job Categories**

  Job postings are also summarized by job category, providing an overview of the types of technical roles represented in the dataset.

- **Company Hiring Activity**

  The dashboard includes a ranking of companies based on the number of job postings represented in the curated dataset. This provides a high-level view of organizations with multiple technical job postings in the collected data.

14. DOWNSTREAM APPLICATIONS

14.3 INTERACTIVE FILTERING

The interactive nature of the dashboard allows users to filter and explore the data based on geographic location. Specifically, a dedicated city filter enables users to transition smoothly from a macro, nationwide overview of the Saudi technical job market to localized insights for specific cities.

14.4 POTENTIAL FUTURE APPLICATIONS

In addition to the implemented dashboard, the curated dataset provides a foundation for several potential downstream applications.

- **Job Search**

  The structured dataset could support a job-search application where users can search and filter job opportunities based on skills, location, company, seniority level, education, employment type, or job category.

- **Job Recommendations**

  The dataset could be extended to support job recommendation systems. A recommendation application could compare the skills and attributes associated with job postings with a user's skills and preferences to identify potentially relevant opportunities.

- **Job Market Analytics**

  The curated dataset can also support additional job-market analysis, including comparisons of skill demand across cities, company hiring activity, job-category distributions, and changes in demand over time where sufficient historical data is available.

- **AI-Powered Applications**

  The structured dataset could serve as a foundation for future AI-powered applications, such as natural-language job search, automated job-market question answering, job summarization, or personalized career insights.

15. KNOWN LIMITATIONS

- **Residual Duplicate URLs:** A small number of duplicate URLs (approximately **7** detected) remain in the processed dataset. These may result from title-text variations across multiple crawls of the same job posting. Further refinement of the deduplication logic may be required to identify and consolidate such records.
- **dbt-utils Deprecation Warnings:** dbt-utils generic tests currently generate deprecation warnings due to the existing test syntax. These warnings are non-blocking and do not affect pipeline execution, but the syntax should be updated in a future maintenance cycle.
- **Seed Schema Inconsistency:** The skill_synonyms seed currently loads into the PUBLIC schema instead of the intended staging_dbt schema. This does not affect the seed's functionality, but it creates an inconsistency with the project's schema organization convention and should be corrected for better maintainability.
- **Limited Historical Tracking:** The current pipeline does not maintain a complete historical record of changes to job postings, such as changes in job descriptions, skills, or employment type over time. Therefore, reposts or modifications to existing postings may not be fully represented historically.

16. SUMMARY OF THE RESULT

Overall, the pipeline transformed fragmented job-posting data from multiple sources into a curated and reusable analytical dataset. The resulting dataset contains **1,331** unique job postings, **707** companies, **15** locations, **723** distinct technical skills, and **7,883** job-skill relationships.

The dataset has already been consumed by an interactive **Power BI** dashboard and can also serve as a foundation for future search, recommendation, analytical, and AI-powered applications.